
用户信息管理平台

安全漏洞挖掘与修复报告

<http://10.133.25.94:5000>

项目版本：V7.0 - 文件包含漏洞(LFI/RFI)专题

报告日期：2026年7月11日

今日课程：Day5:文件包含漏洞深度解析

技术栈：Python Flask / LFI / RFI / 路径遍历

一、课程背景

文件包含漏洞(File Inclusion)是Web安全中的经典漏洞类型，分为LFI(本地文件包含)和RFI(远程文件包含)两种。LFI允许攻击者读取服务器上的任意文件，RFI则允许攻击者远程加载并执行恶意代码。该漏洞通常出现在动态加载页面、模板渲染等功能中。

1.1 LFI vs RFI

类型	全称	特点	危害
LFI	本地文件包含	读取服务器本地文件	读取系统文件、日志投毒
RFI	远程文件包含	加载远程服务器文件	远程代码执行、获取shell

1.2 本次实验环境

项目	说明
目标应用	基于Flask的用户管理系统
新增功能	/page?name=X 动态页面加载
漏洞类型	LFI路径遍历漏洞
测试账号	admin/admin123

二、漏洞分析

2.1 漏洞代码

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    # 直接拼接用户输入的 name 到路径中
    file_path = os.path.join("pages", name)
    # 不校验 ../
    with open(file_path, "r") as f:
        content = f.read()
```

2.2 POC 1: LFI 读取系统文件

```
/page?name=../../etc/passwd    -> 读取系统密码文件
/page?name=../../etc/hostname  -> 读取主机名
/page?name=../app.py           -> 读取源代码
```

2.3 POC 2: 正常访问帮助中心

```
/page?name=help -> 显示帮助中心页面
```

三、修复方案

3.1 路径白名单校验

```
ALLOWED_PAGES = {"help", "about", "contact"}
if name not in ALLOWED_PAGES:
    return "页面不存在"
```

3.2 路径规范化校验

```
real_path = os.path.realpath(os.path.join(PAGES_DIR, name))
if not real_path.startswith(os.path.realpath(PAGES_DIR)):
    return "非法路径"
```

3.3 修复前后对比

对比项	修复前	修复后
路径处理	直接拼接	白名单/规范化校验
../过滤	无过滤	检查是否越界

四、总结

文件包含漏洞的核心原因是对用户输入的路径参数缺乏校验。攻击者通过 `../` 跳出限制目录实现 LFI。

安全开发原则：

1. 永远不要直接将用户输入拼接到文件路径中
2. 使用白名单限制可访问的文件
3. 使用 `os.path.realpath()` 规范化路径后校验范围

— 报告完 —

报告日期: 2026-07-11 | 课程: 文件包含漏洞 LFI/RFI